Lecture 3

OO Modelling, **UML Static Diagrams** (**Class**, **Object**, & **Package** **Diagrams**)

Advanced system analysis

DR.Nesma Ebrahim

TOPICS COVERED

- Continuing Class Diagrams
- Associations
- Multiplicity
- Links and Associations
- Roles
- Self-Associations
- Association classes vs. Ordinary classes
- Qualified Association
- A sample class model
- Aggregation & Composition Relation
- Generalization, Specialization & Inheritance
- Overriding Features
- Abstract Class
- Object Diagrams
  - Elements & An Example
- Class Modelling Tips
- Divide and Conquer: Modularization
- Cohesion
  - Types of Cohesion
  - Coincidental Cohesion
  - Functional Cohesion
- Coupling
  - Examples
  - Consequences of Coupling
- Package Diagrams

ASSOCIATIONS

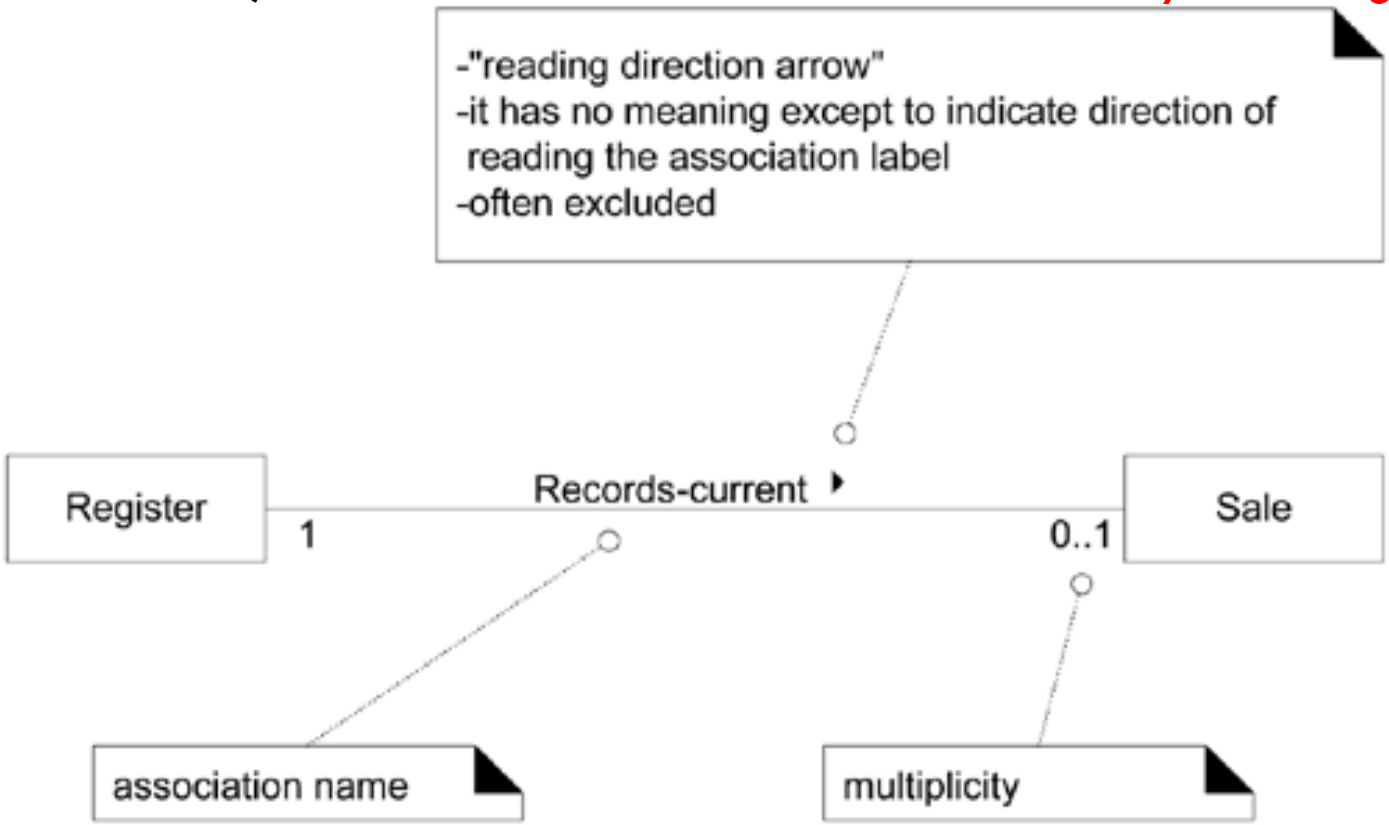
An association is a relationship between classes that indicate some meaningful and interesting relationship.

It's represented by a line with a name.

Properly naming associations is important to enhance understanding: use verb phrase.

ASSOCIATIONS

An optional "reading direction arrow" indicates the direction to **read** the association name; *it does not indicate direction of visibility or navigation*



ASSOCIATIONS

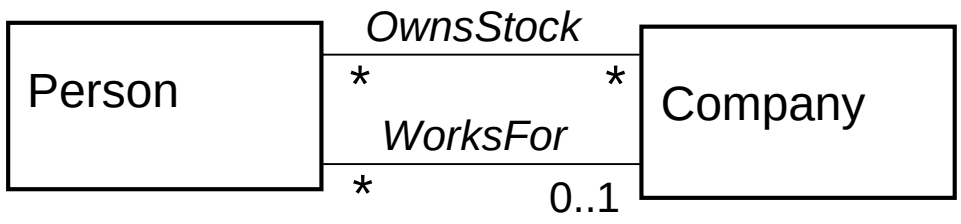
There may be more than one associations between classes (this is not uncommon).

Uni-directional association:

association can access only instnaces in a class. More easy in implementation and maintainance.

Bi-directional association: can acess any refrenceses in class

More complex in Mainatinance and implementation



ASSOCIATIONS

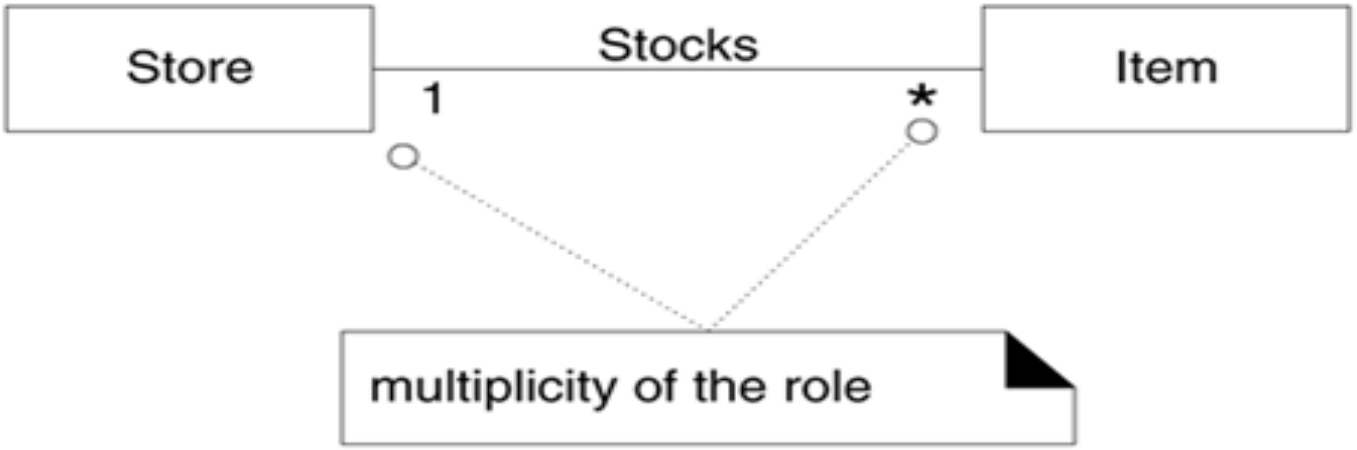
Associations are usually implemented by a ***reference*** from an object to another.(**Bi-directional association) defult.**

Associations are inherently bidirectional. They can be traversed in either direction. A person *WorksFor* a company and a company *Employs* a person.

Associations could be unidirectional.

MULTIPLICITY

Multiplicity specifies the number of instances of one class that may relate to a single instance of an associated class.

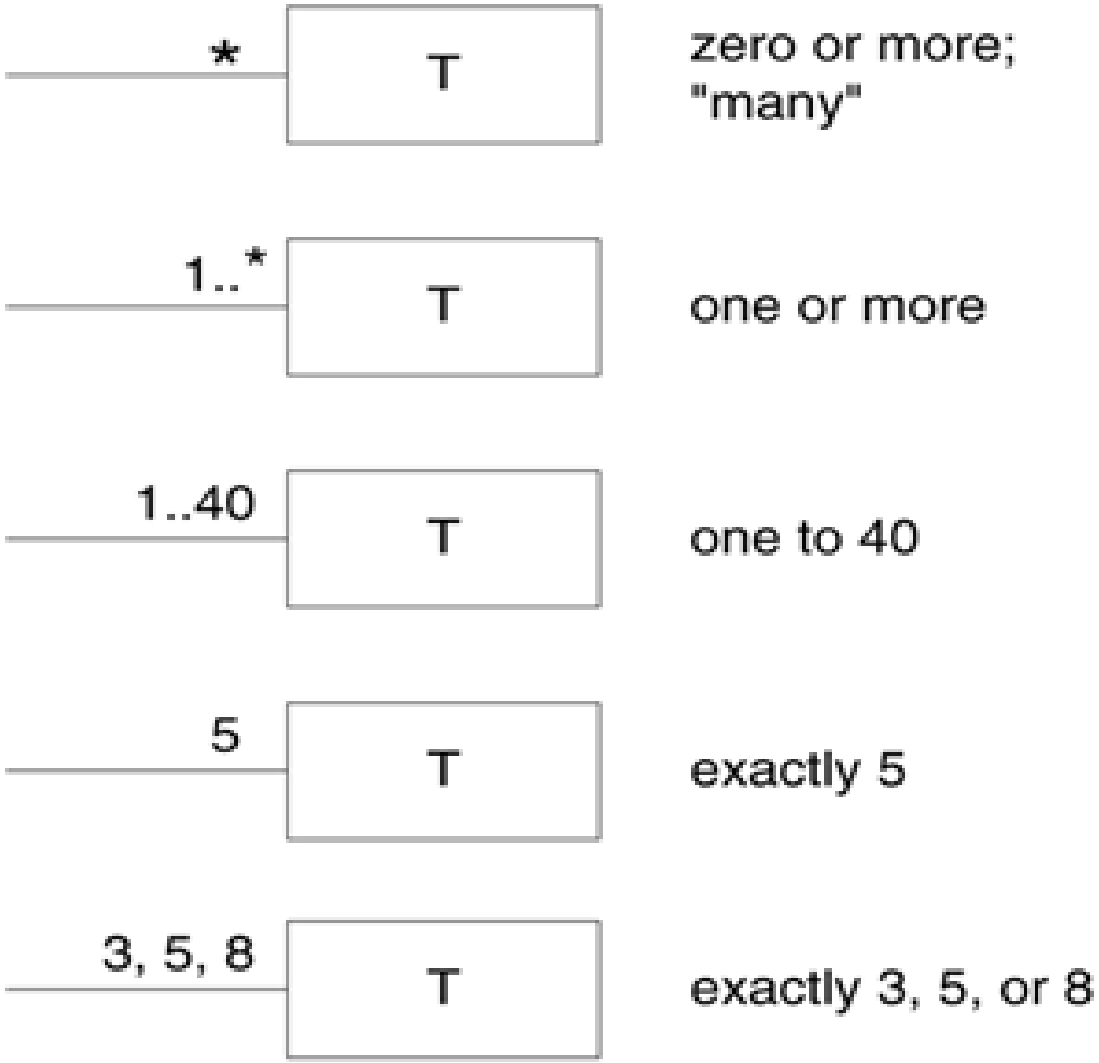


MULTIPLICITY

Multiplicity exposes hidden assumptions in the model ..

For example, if a person ***WorksFor*** a company, can he work for more than one company? In other words, is it ***one-to-one*** or ***one-to-many*** association?

MULTIPLICITY VALUES



LINKS AND ASSOCIATIONS

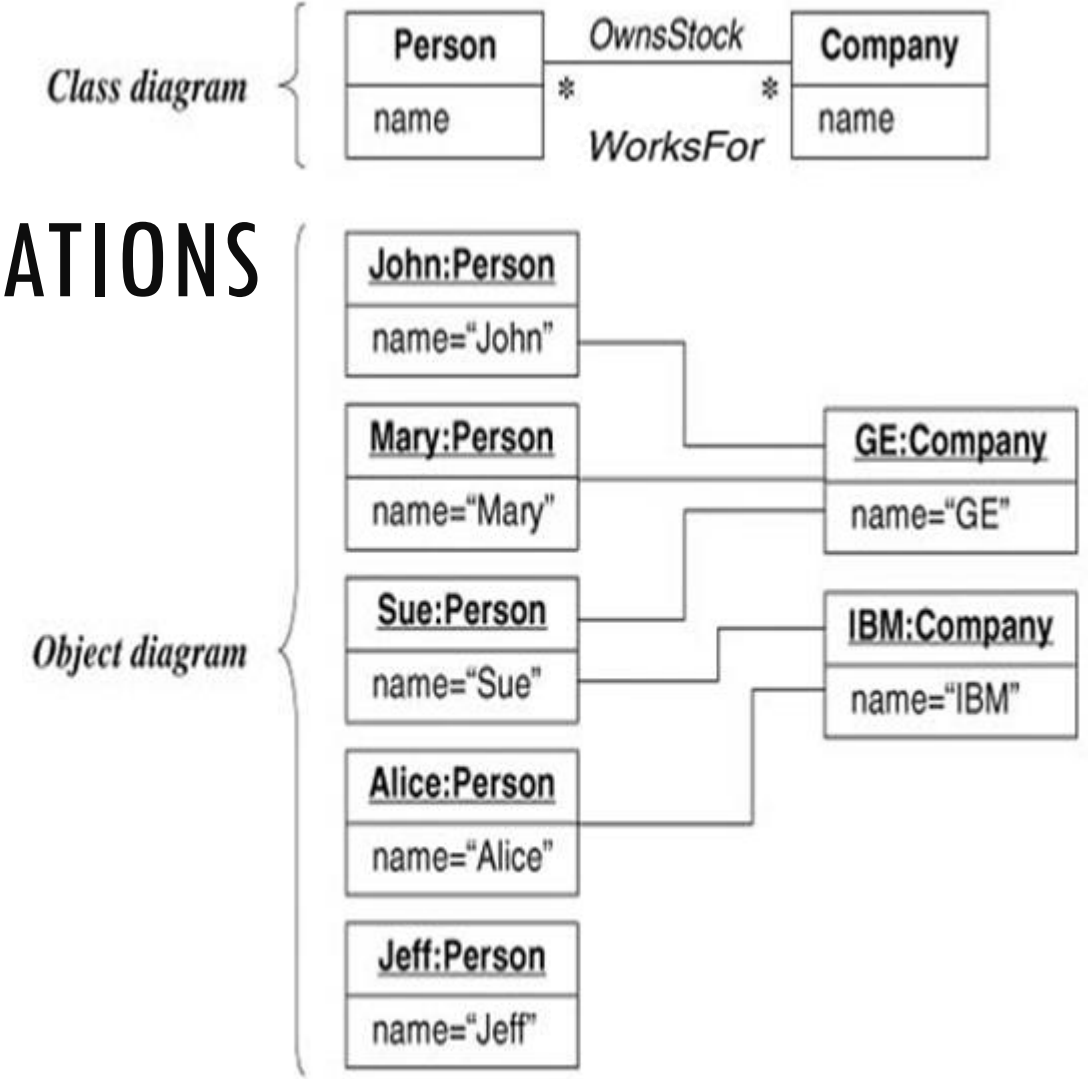


Figure 3.7 **Many-to-many association.** An association describes a set of potential links in the same way that a class describes a set of potential objects.

A ***link*** is a physical or conceptual connection among objects

- For example John works for GE company.

Links and objects can broken and temporarly

An ***association*** is a relationship between classes and represents group of links.

- For example, a person *WorksFor* a company.

Associations and classes cant destroyed and it permenent

LINKS AND ASSOCIATIONS

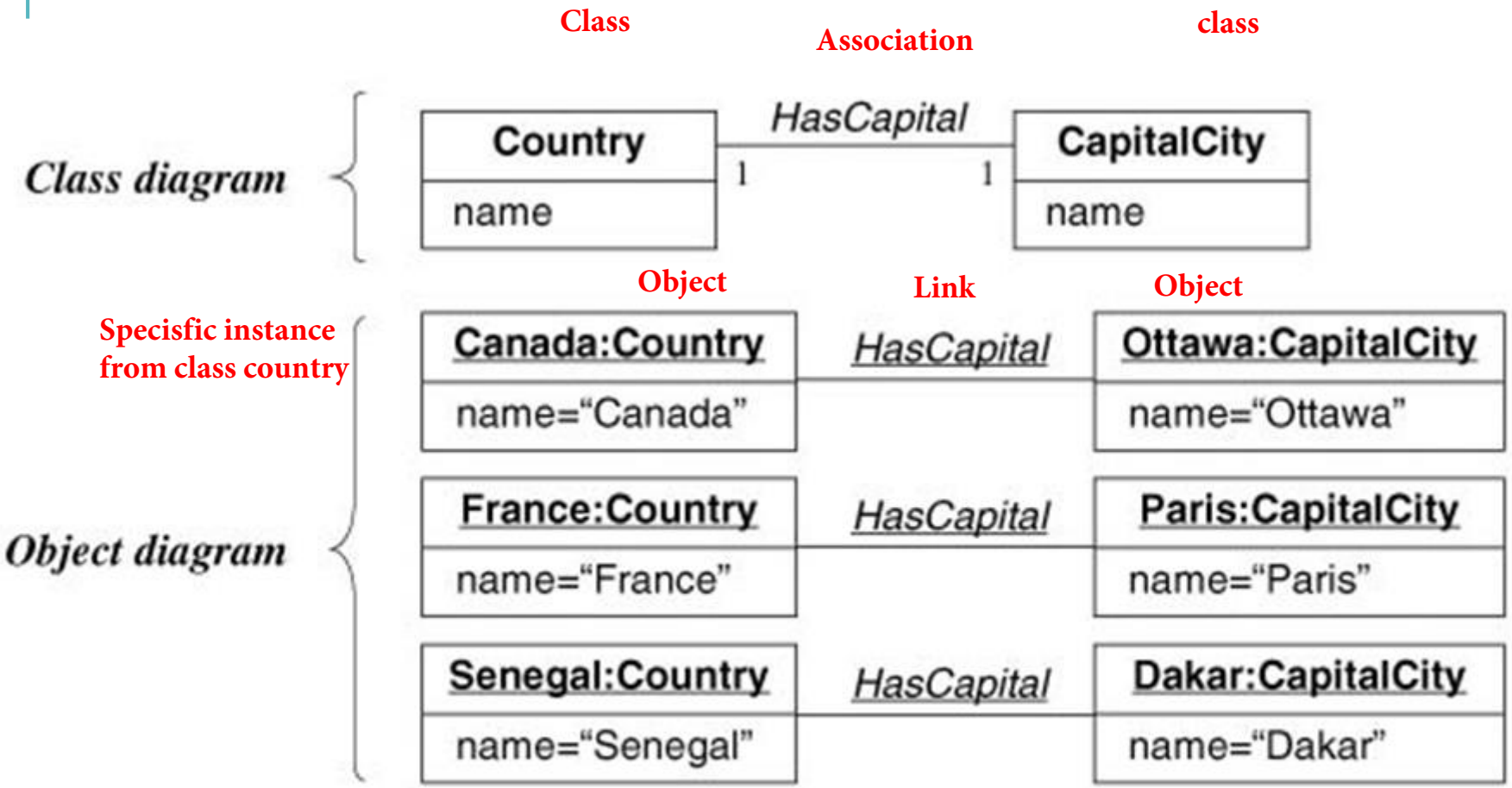


Figure 3.8 One-to-one association. Multiplicity specifies the number of instances of one class that may relate to a single instance of an associated class.

LINKS AND ASSOCIATIONS

LINKS AND ASSOCIATIONS

classes has more than associations so, objects has more than one link

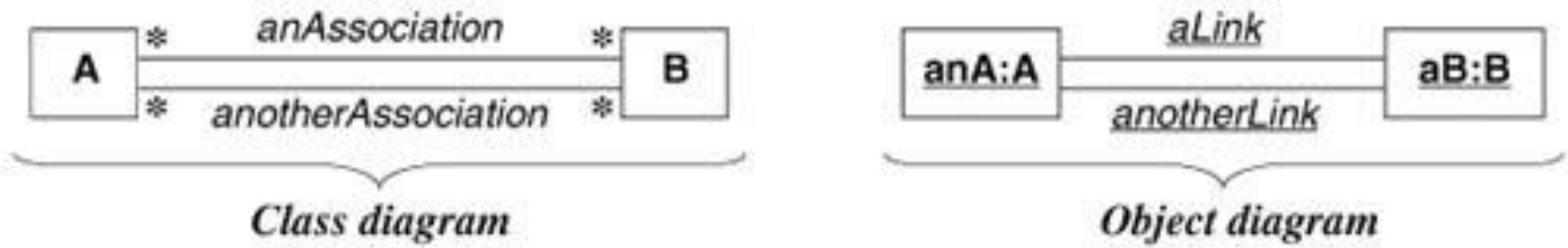


Figure 3.11 Association vs. link. You can use multiple associations to model multiple links between the same objects.

ROLES

Each association End can be labeled by a role.

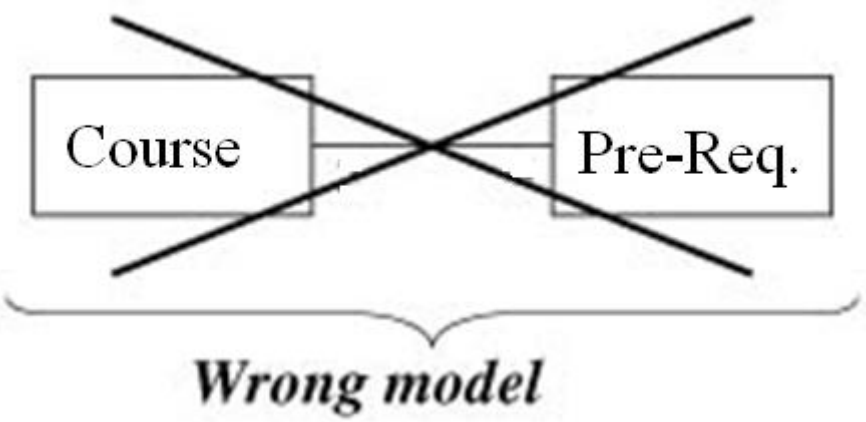
This makes **understanding** associations easier.

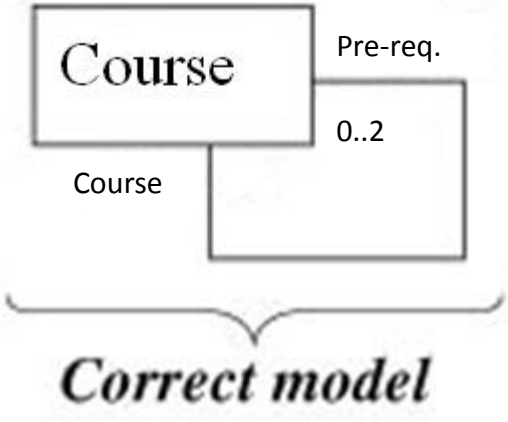
They are **especially important for** ***Self-associations*** between objects of the same class.

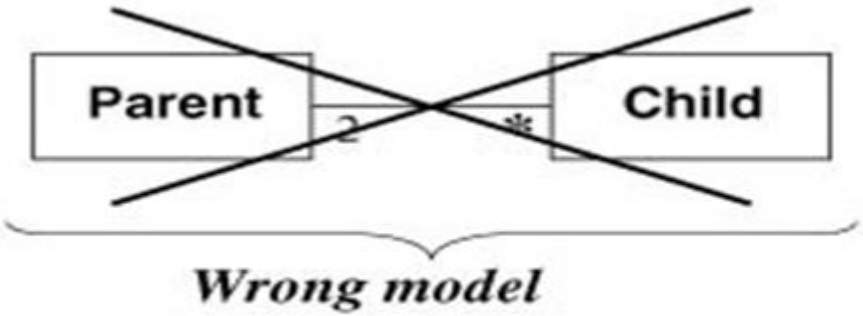
employee	employer
Joe Doe	Simplex
Mary Brown	Simplex
Jean Smith	United Widgets

Figure 3.12 Association end names. Each end of an association can have a name.

SELF ASSOCIATIONS (RECURSIVE-ASSOCIATIONS)







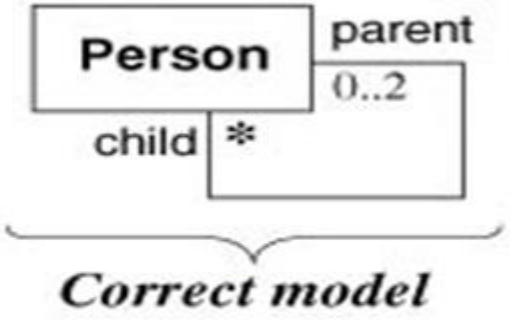


Figure 3.14 Association end names. Use association end names to model multiple references to the same class.

ASSOCIATION CLASSES

An ***Association class*** is an association that is also a class (has attributes and operations)

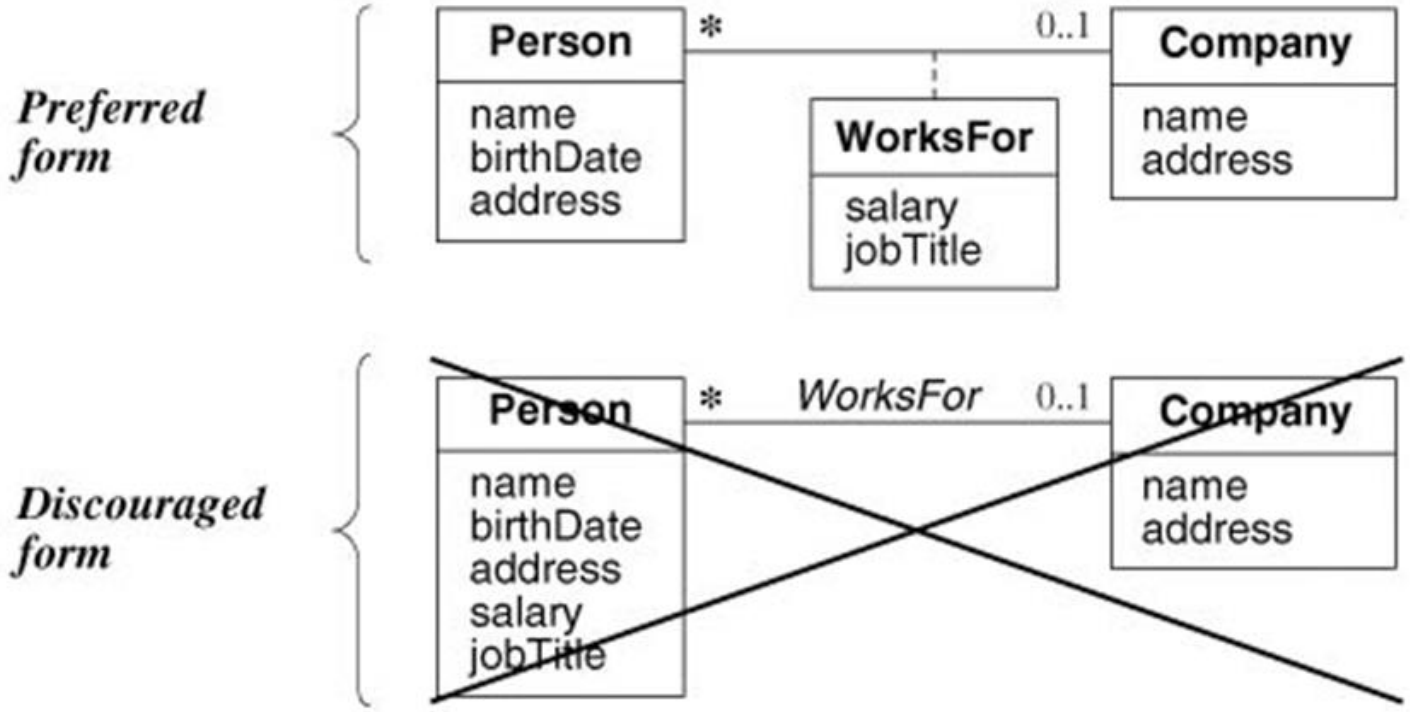


Figure 3.19 Proper use of association classes. Do not fold attributes of an association into a class.

ASSOCIATION CLASSES

Many-to-may associations provide compelling rationale for association classes.

accessPermission is a joint property of File and User and cannot placed in one of them only.

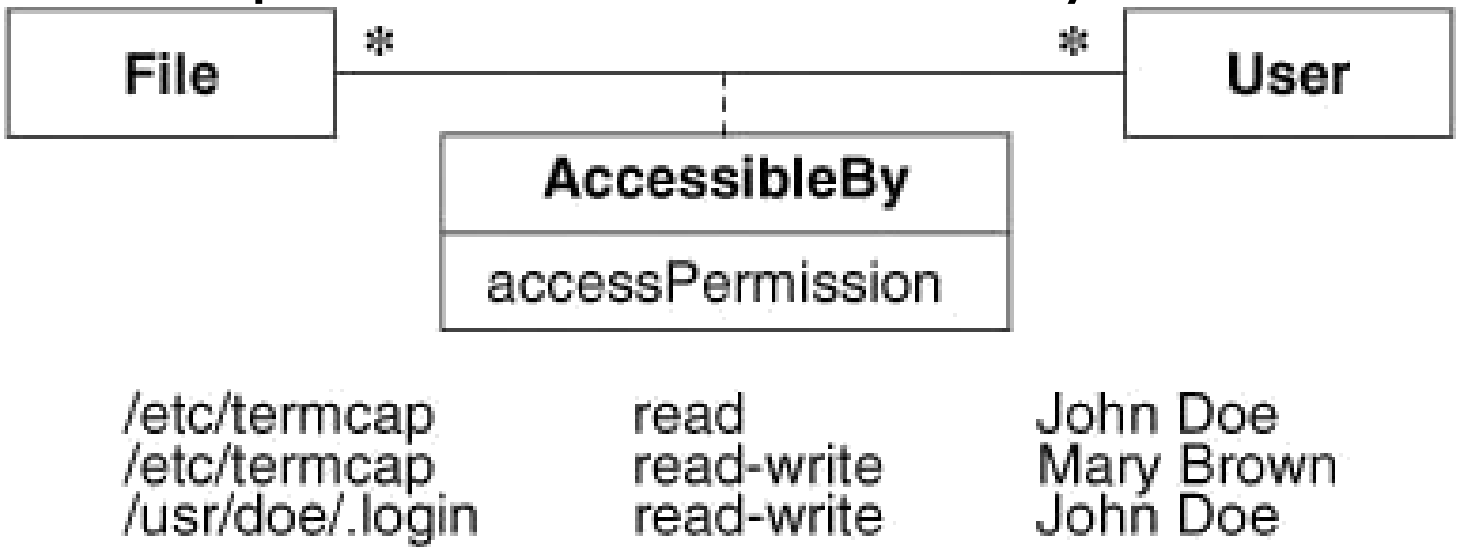


Figure 3.17 An association class. The links of an association can have attributes.

ASSOCIATION CLASSES VS. ORDINARY CLASSES

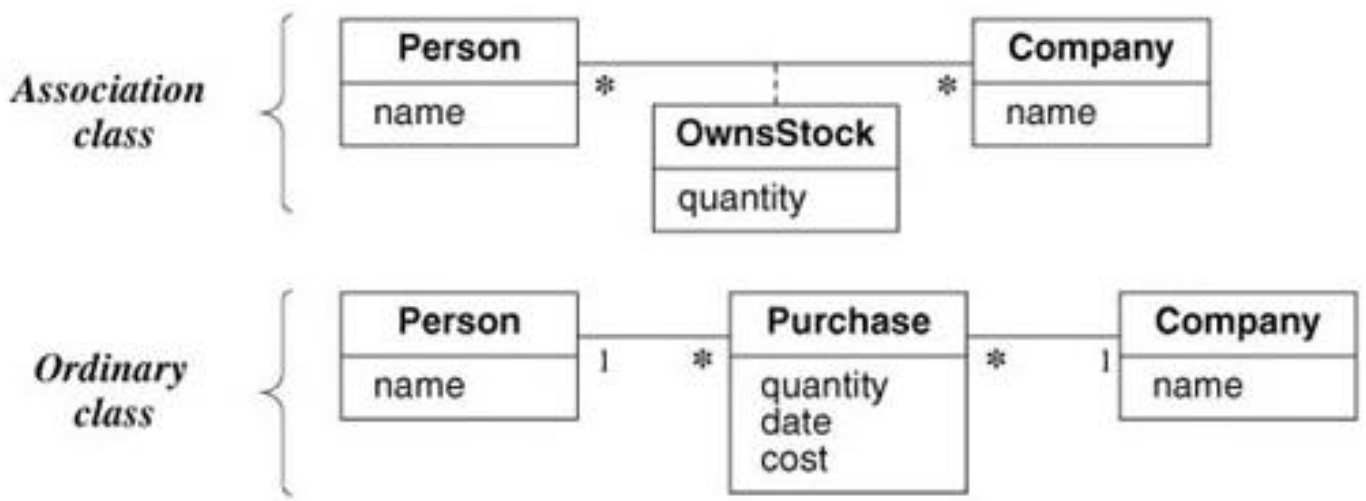


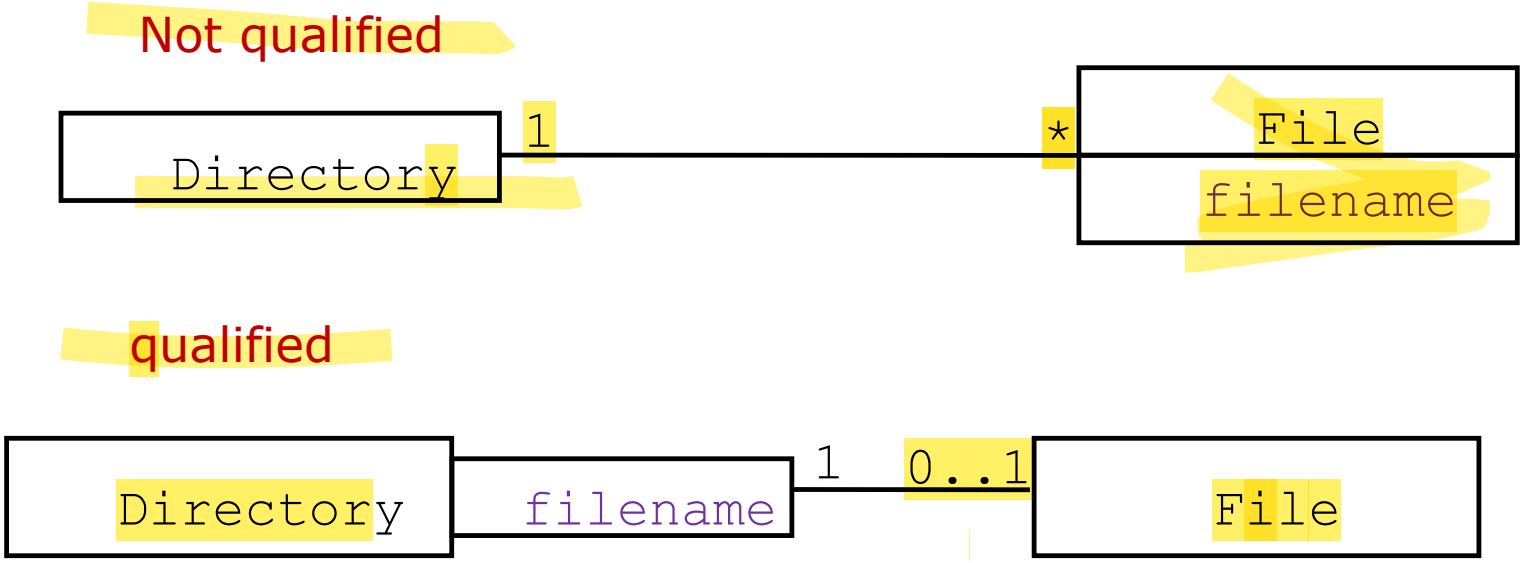
Figure 3.21 Association class vs. ordinary class. An association class is much different than an ordinary class.

Association class *OwnsStock* has only one occurrence for each pair of *person* and *company* (i.e. only one link exists between pair of person and company objects).

In contrast there can be any number of *purchase* objects for each pair of *person* and *company* objects.

QUALIFIED ASSOCIATION

A ***qualified association*** is an association in which an attribute called the ***qualifier*** is used to reduce a *many* relation to *one* relation on the other end.



QUALIFIED ASSOCIATION

A bank has many accounts [0..*]

But a bank + account number gives only one account

Account Number is attribute of account class.

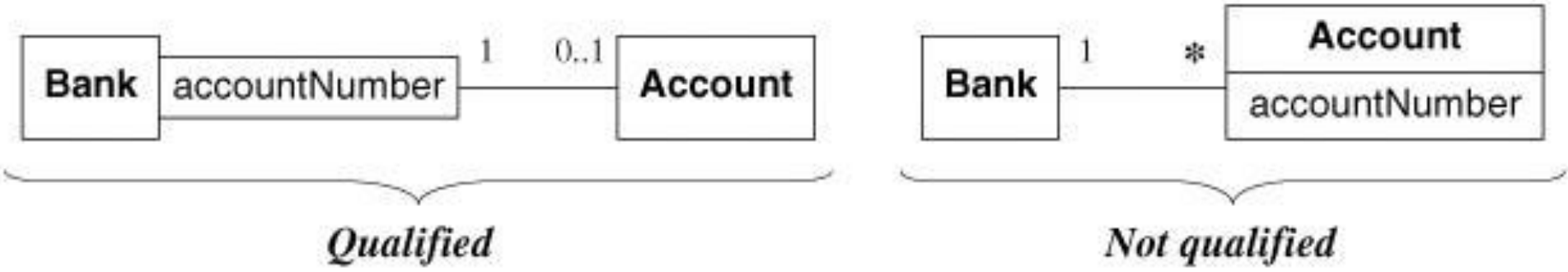


Figure 3.22 Qualified association. Qualification increases the precision of a model.

The order of modeling is:

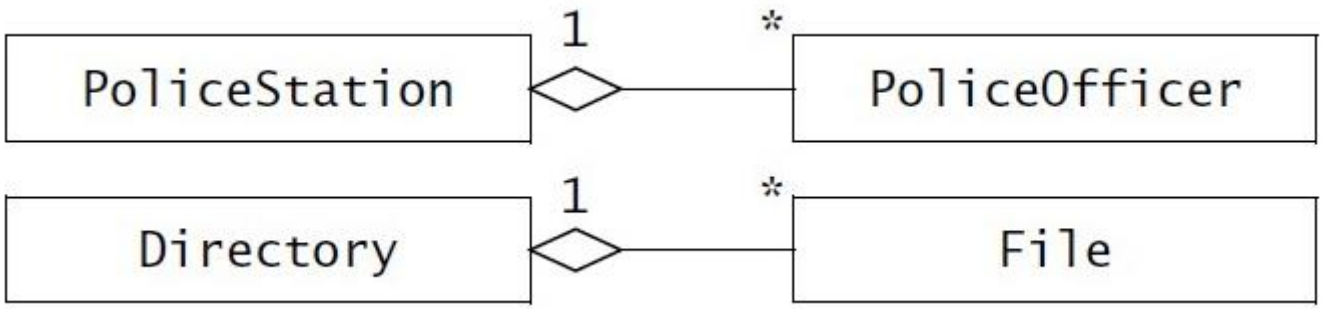
- Define classes
- Define associations
- Define multiplicity

A SAMPLE CLASS MODEL

Add some suitable attributes and operations to the classes of the flight system.

FURTHER CLASS CONCEPTS
AGGREGATION RELATION

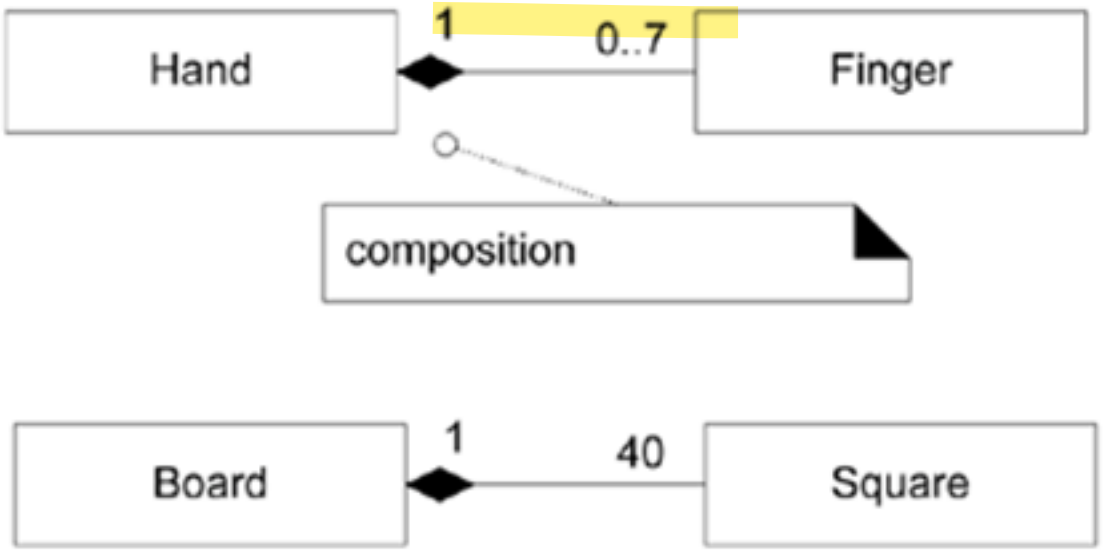
Aggregation is a special kind of association where an aggregate object contains constituent parts. It's *has-a* relationship



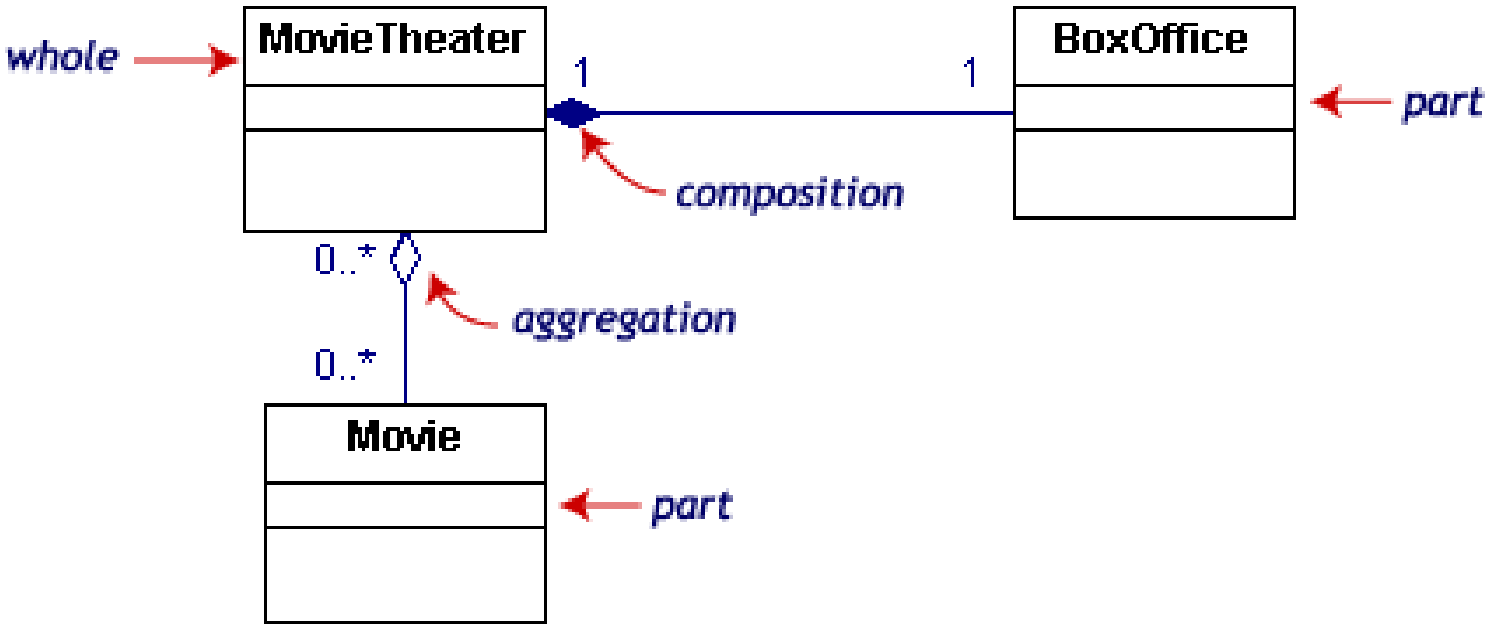
FURTHER CLASS CONCEPTS

COMPOSITION RELATION

Composition is a special kind of association in which a constituent part belongs at most to one assembly and exists only if the assembly exists.



FURTHER CLASS CONCEPTS
AGGREGATION & COMPOSITION RELATION



FURTHER CLASS CONCEPTS
GENERALIZATION, SPECIALIZATION AND INHERITANCE

Generalization is the relationship between class (the superclass) and one or more variations of the class (the subclass).

The superclass holds common attributes, operations and associations.

The subclasses add specific attributes, operations and associations.

Each subclass is said to ***inherit*** the features of the its superclass.

Inheritance is called "***is-a***" relationship.

GENERALIZATION, SPECIALIZATION AND INHERITANCE

The terms ***generalization***, ***specialization*** and ***inheritance*** refer to aspects of the same idea, they are used in place of one another .

Generalization refers to the super-class generalizing its subclasses

Specialization refers to the fact that the subclass specializes or refines the super-class,

Inheritance refers to the **mechanism** for implementing *generalization / specialization*.

GENERALIZATION, SPECIALIZATION AND INHERITANCE

Generalization is ***transitive*** across an arbitrary number of levels.

An ***ancestor*** is parent or grandparent class of a class.

A ***descendant*** is a child or grandchild of a class.

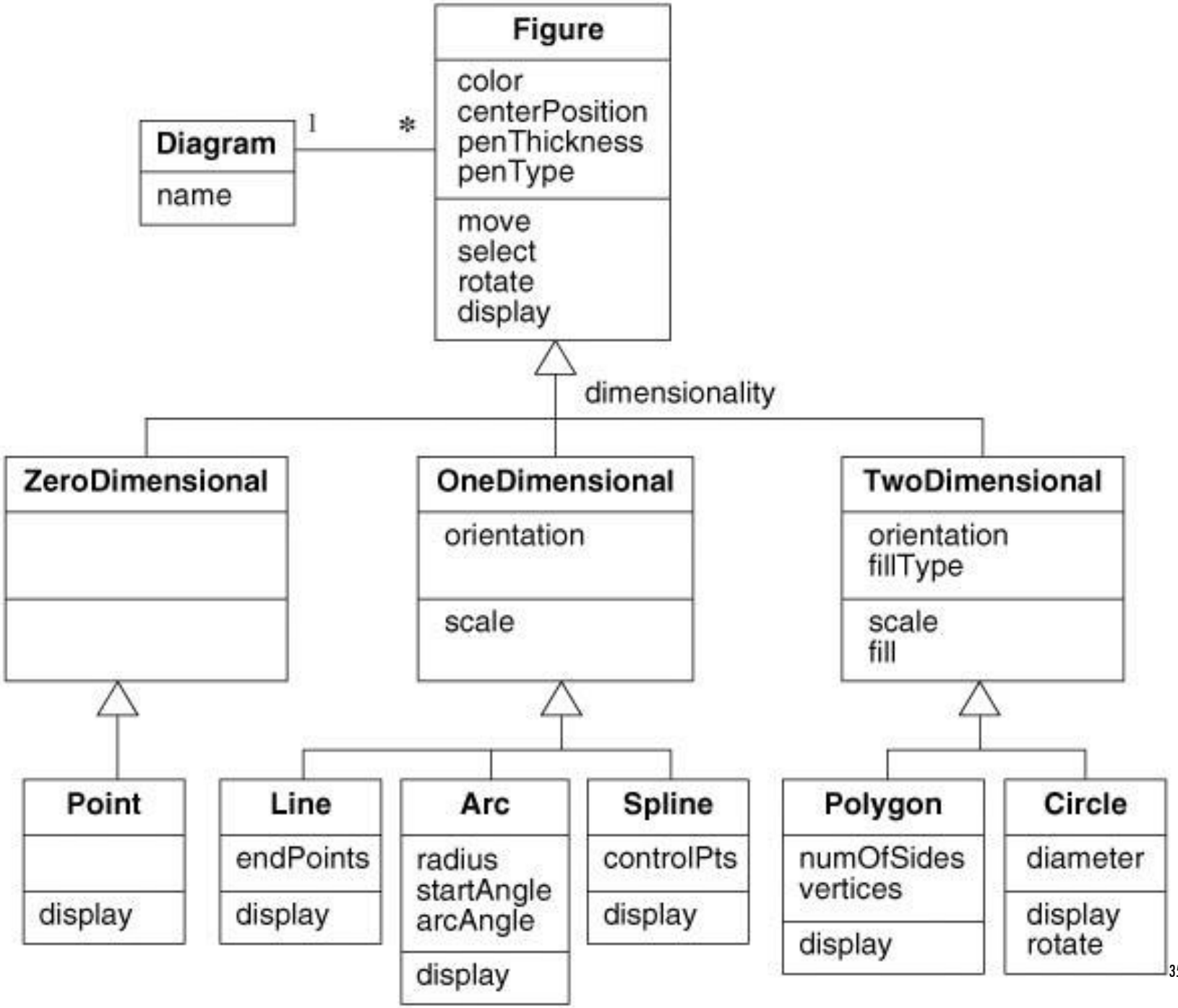


Figure
color
centerPosition
penThickness
penType
move
select
rotate
display
Diagram
name
1
*
dimensionality
ZeroDimensional
OneDimensional
orientation
scale
TwoDimensional
orientation
fillType
scale
fill
Point
display
Line
endPoints
display
Arc
radius
startAngle
arcAngle
display
Spline
controlPts
display
Polygon
numOfSides
vertices
display
Circle
diameter
display
rotate

USE OF GENERALIZATION

Generalization serves three purposes.

1. ***Supporting polymorphism***.
2. ***Structuring the description of objects*** and their relation to each other based on their similarity and differences.
3. ***Reusing code***. You inherit code from super-classes or from a class library automatically with inheritance. Reuse is more productive than repeatedly writing code form scratch. **When reusing code, you can adjust the code if necessary to get the exact desired behavior**.

OVERRIDING FEATURES

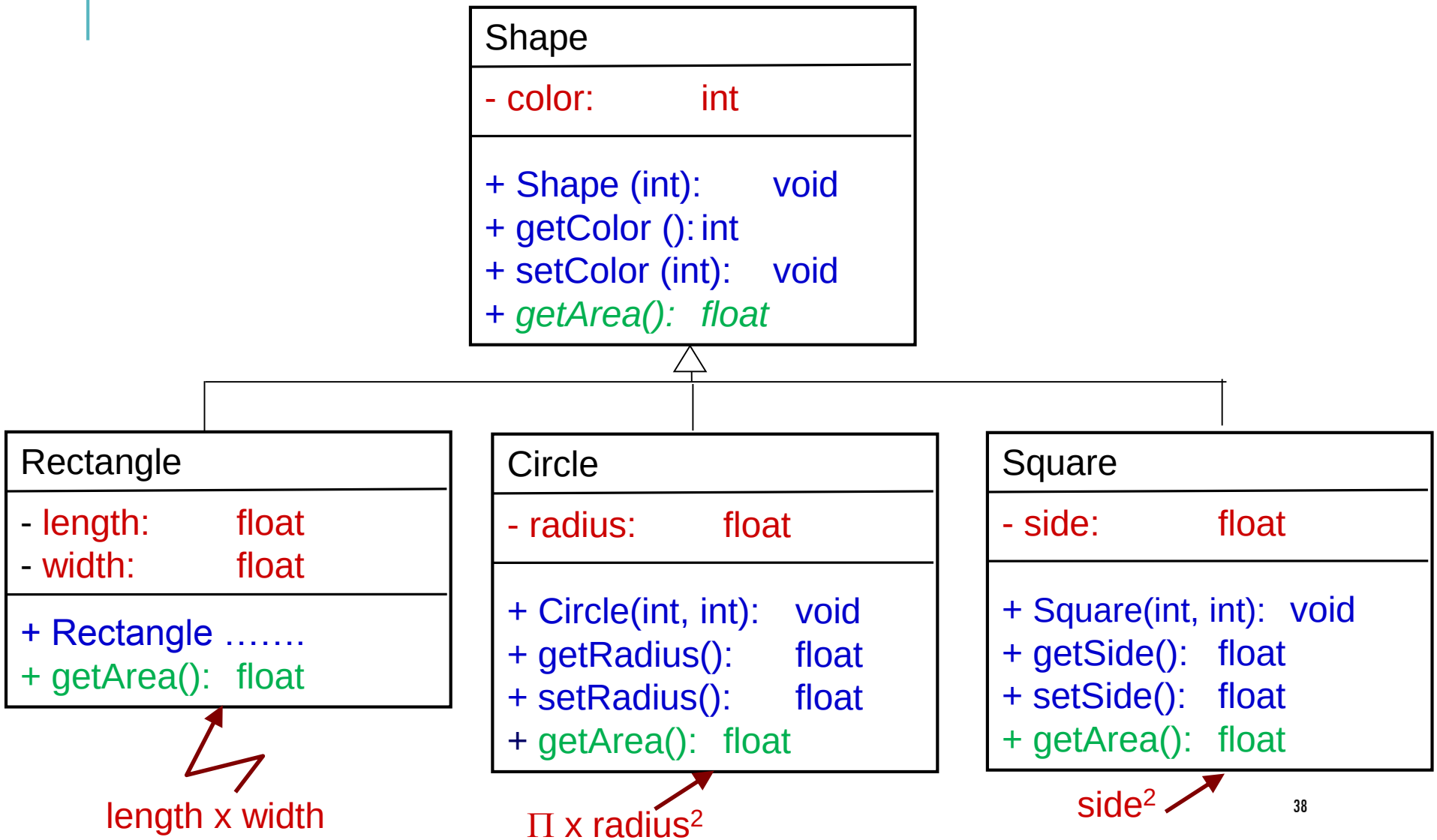
A subclass may ***override*** a superclass feature by defining a feature with the same name.

The overriding feature (in the sub-class) replaces the overridden feature (in the super-class)

Never override a feature so that it is inconsistent with the original inherited feature.

Overriding should preserve the attribute type, number and type of arguments of an operation and its return type.

GENERALIZATION, SPECIALIZATION AND INHERITANCE



FURTHER CLASS CONCEPTS
ABSTRACT CLASS

An incomplete class that cannot be instantiated.

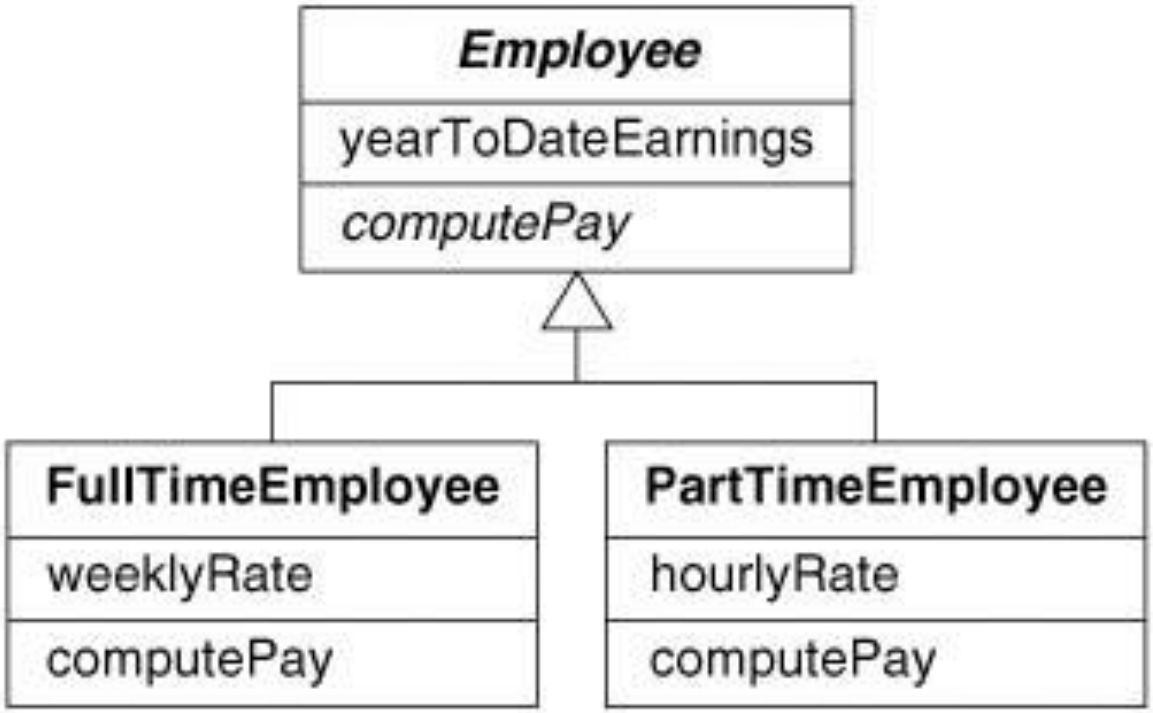


Figure 4.13 Abstract class and abstract operation. An abstract class is a class that has no direct instances.

OBJECT DIAGRAMS

An object model: is a collection of objects, where every object in the model is an instance of a class in the class model.

- it represents a snapshot of the system at one instant in time.
- no object can have attributes or relationships that are not in the class model.

An object model may be thought of as an instantiation of a class model.

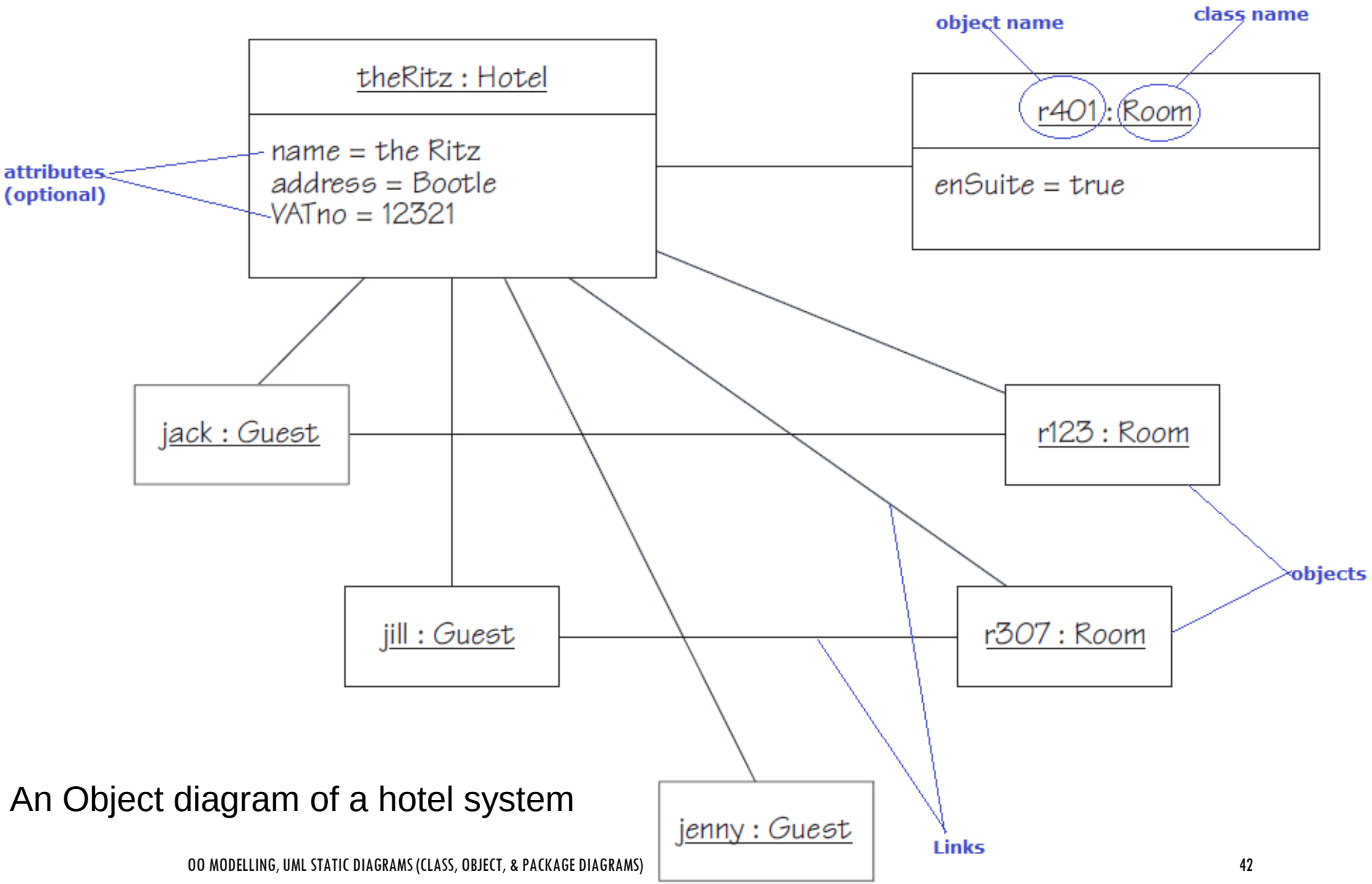
- It shows concrete instances of relevant classes & relationships.
- It indicates what connections do exist at that instant in time.
- Is visually represented using an object diagram.

OBJECT DIAGRAMS .. ELEMENTS

An **object diagram**: is a visual representation of an object model, showing a snapshot of the system at some point in time. The elements of this object diagram are:

- Rectangular boxes representing **objects**.
  - Inside each box you add:
    - ObjectName : ClassName which are underlined.
    - Names and values of useful attributes in a second compartment (Optional)
  - Lines drawn between objects are called **links**.
    - They represent particular cases of one object 'knowing about' another object.
    - Each link is an instance of an association (a relationship between classes).

OBJECT DIAGRAMS .. AN EXAMPLE



An Object diagram of a hotel system

CLASS MODELING TIPS

The following categories are useful sources of relevant objects:

- **Tangible objects**: the physical things in the domain such as rooms, bills, books and vehicles;
- **Roles**: the roles played by people in the domain, such as employees, guests and members;
- **Events**: the circumstances, episodes, interactions, happenings and significant incidents, such as room reservations, vehicle registrations, orders, deliveries and transactions;
- **Organizational units**: the groups to which people belong, such as accounts departments, production teams and maintenance crews.

CLASS MODELING TIPS

Scope: A model represents the relevant aspects of the problem. *Exercise judgment in deciding which objects to show and which ones to ignore.*

Simplicity: *Make the model as simple as possible.* Simpler models are easier to understand and develop.

Diagram Layout: *Draw your diagram in a way that is easy to understand*. Avoid cross lines. Position important classes where they are visually prominent.

Names: *Carefully choose class names*. Choose descriptive names and use singular nouns to name classes.

DIVIDE & CONQUER: MODULARIZATION

The only way till now to reduce the complexity of systems is **modularization**.

Modularization means decompose system or problem to a smaller sub-system with separate boundaries (modules).

Examples of modules are:

- Whole programs or applications;
- Software libraries;
- Procedures & functions in classical PLs;
- Classes, in an OO PL such as Java.

DIVIDE & CONQUER: MODULARIZATION

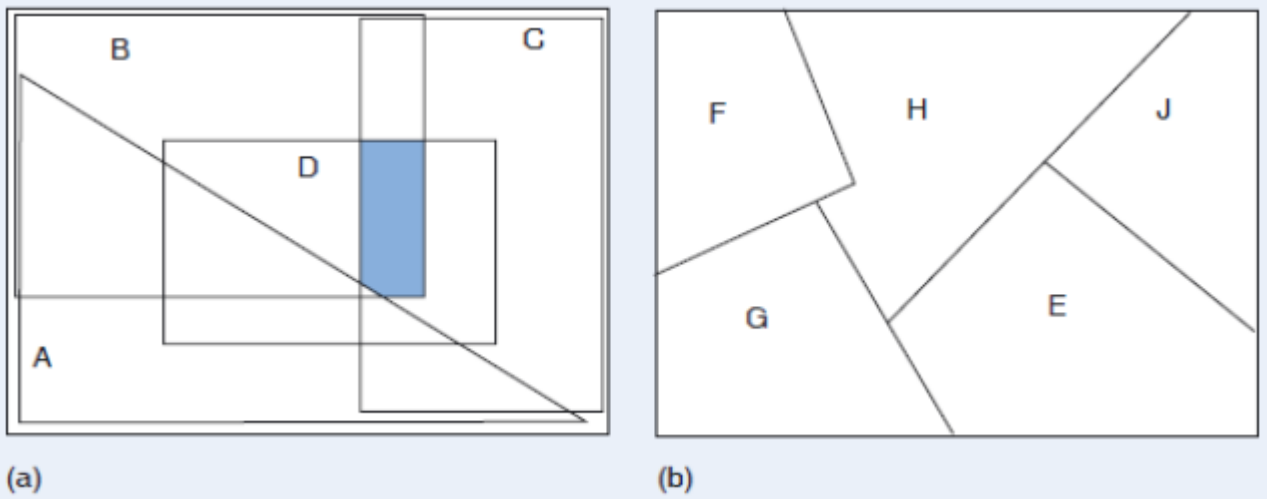
Modules in themselves are not "good" ..

Must design them to have good properties ..

Good modularization must have:

- Maximal relationships within modules (***Cohesion***)
- Minimal relationships between modules (***Coupling***)

DIVIDE & CONQUER: MODULARIZATION



There are two forms of decomposition:

(a) Projection: dependent modules, common elements between modules.

(b) Partitions: independent modules, which are easy to use.

Note: Usually even if the modules sound separate there are relationships between them which must be taken into consideration through the software development and during maintenance.

COHESION

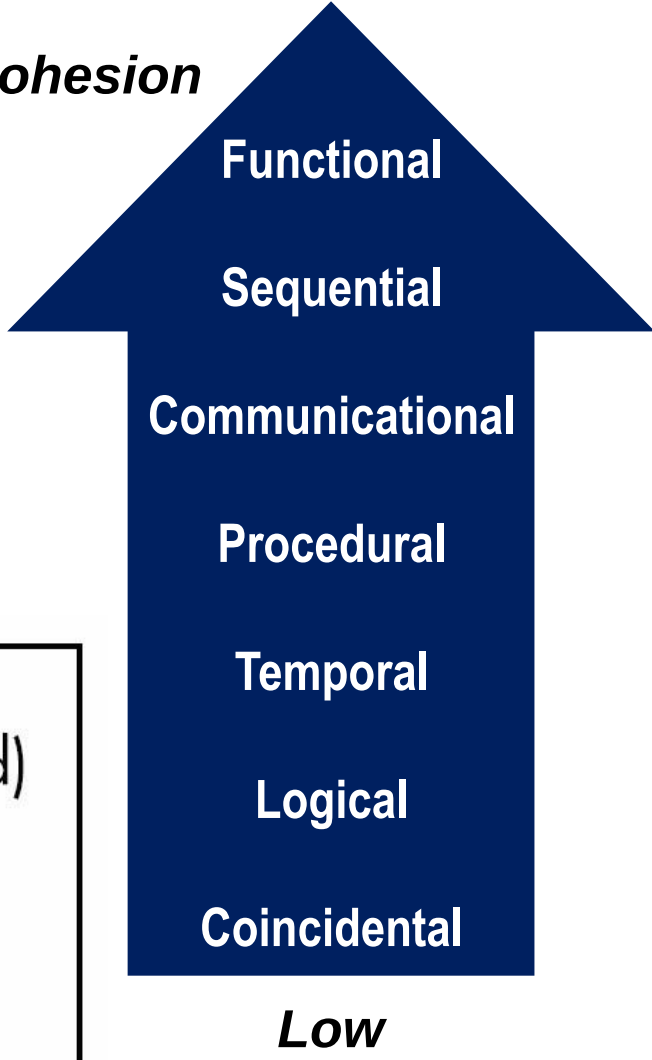
Definitions:

- *The degree to which all elements of a component are directed towards a single task.*
- *The degree to which all elements directed towards a task are contained in a single component.*
- *The degree to which all responsibilities of a single class are related.*

Internal glue with which component is constructed.

All elements of component are directed toward and essential for performing the same task.

TYPES OF COHESION



7.	Informational cohesion	(Good)
	Functional cohesion	
5.	Communicational cohesion	
4.	Procedural cohesion	
3.	Temporal cohesion	
2.	Logical cohesion	
1.	Coincidental cohesion	(Bad)

COINCIDENTAL COHESION

Def: Parts of the component are unrelated (unrelated functions, processes, or data) .. Module performs multiple, completely unrelated actions ..

- Parts of the component are only related by their location in source code.
- Elements needed to achieve some functionality are scattered throughout the system.
- Accidental
- Worst form

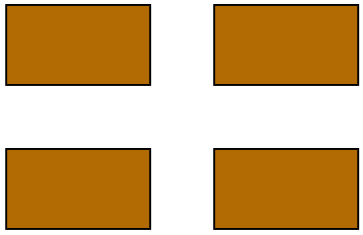
FUNCTIONAL COHESION

Def: Every essential element to a single computation is contained in the component .. Module performs exactly one action ..

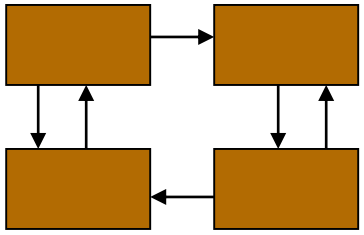
- Every element in the component is essential to the computation.
- Ideal situation.
- What is a functionally cohesive component?
  - One that not only performs the task for which it was designed, but ..
  - it performs only that function and nothing else.

COUPLING

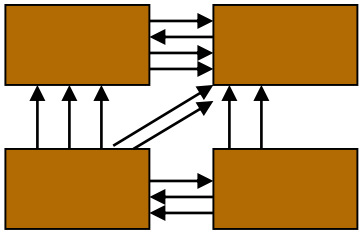
- The degree of dependence such as the amount of interactions among components.



No dependencies



Loosely coupled
some dependencies



Highly coupled
many dependencies

COUPLING .. EXAMPLES

A component depends on another if a change to one requires a change to another.

Examples:

Circular Dependency; a relation between two or more modules which either directly or indirectly depend on each other to function properly. Such modules are also known as mutually recursive. Suppose we have a class called A which has class B's Object (*in UML terms: A HAS B*). At the same time class B is also composed of an Object of class A (*in UML: terms B HAS A)*. Obviously this represents a circular dependency because while creating an object of A, the compiler must know of B .. On the other hand, while creating an object of B, the compiler must know of A.

Chain Dependency; Module A depends on B and B depends on C (chain of interdependent modules) ..

COUPLING .. EXAMPLES

- One component modifies another.
- More than one component share data such as global data structures.
- Component passes a data structure to another component that does not have access to the entire structure.

CONSEQUENCES OF COUPLING

High coupling

- Components are difficult to understand in isolation.
- Changes in component ripple to others.
- Components are difficult to reuse.
  - Need to include all coupled components.
  - Difficult to understand.

Low coupling

- May incur performance cost.
- Generally faster to build systems with low coupling.

PACKAGE DIAGRAM

- Packages let you organize large models so that they are more understandable.

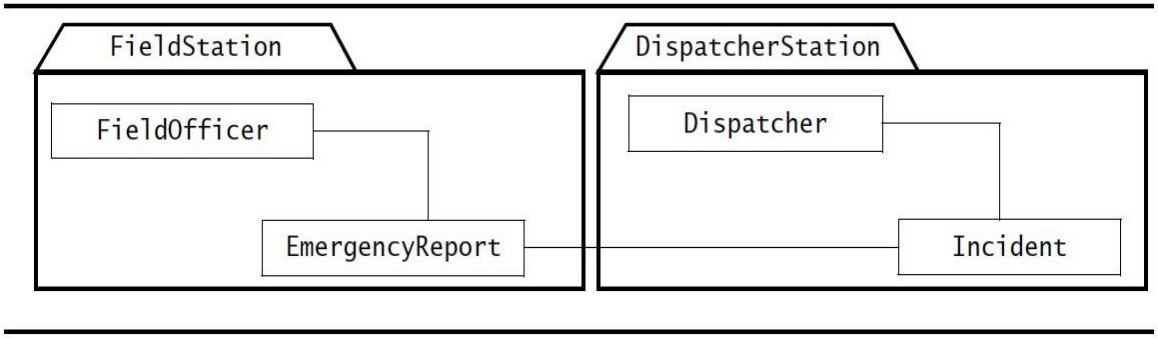
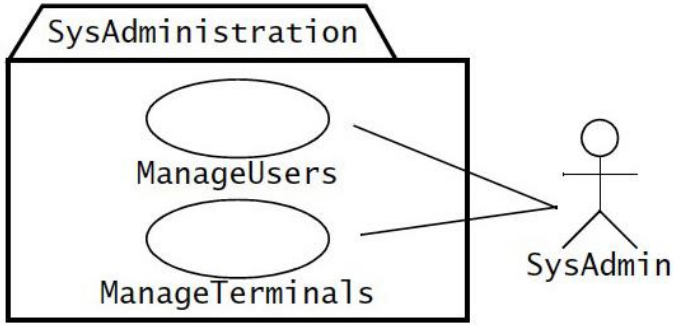
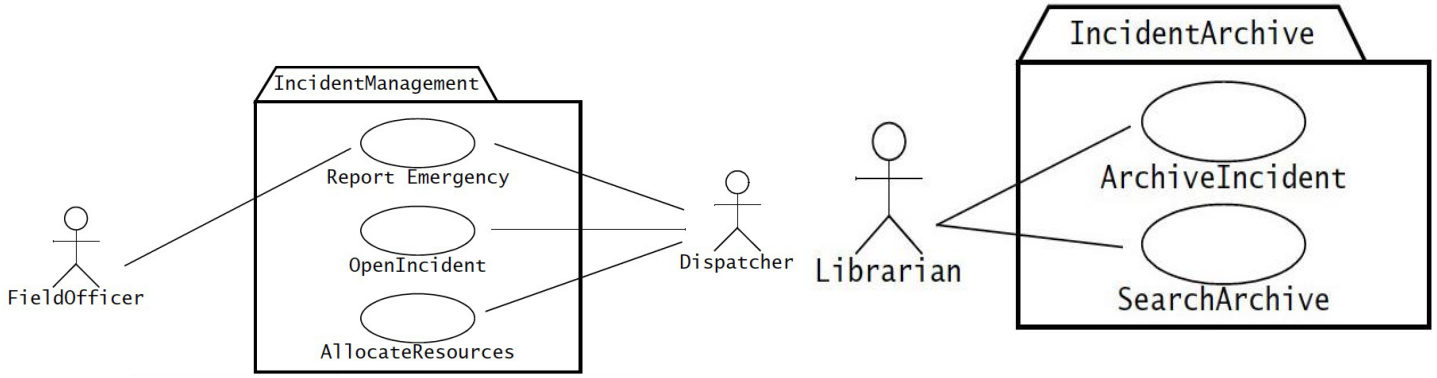


Figure 2-47 Example of packages. The `FieldOfficer` and `EmergencyReport` classes are located in the `FieldStation` package, and the `Dispatcher` and `Incident` classes are located on the `DispatcherStation` package.

PACKAGE DIAGRAM





Example of packages: Use-Cases are organized by actors